

# Часто задавані питання на співбесідах по JavaScript

---

## 1. Типи даних та перевірка типів

Питання: У чому різниця між `==` та `===`?

```
// Нестроге порівняння (==)
console.log(5 == "5");      // true
console.log(0 == false);    // true
console.log(null == undefined); // true

// Строге порівняння (===)
console.log(5 === "5");     // false
console.log(0 === false);   // false
console.log(null === undefined); // false
```

**Пояснення:**

- `==` виконує приведення типів перед порівнянням
- `===` порівнює значення без приведення типів

Питання: Як перевірити, чи є змінна масивом?

```
// Спосіб 1: Array.isArray()
const arr = [1, 2, 3];
console.log(Array.isArray(arr)); // true
console.log(Array.isArray({}));  // false

// Спосіб 2: instanceof
console.log(arr instanceof Array); // true

// Спосіб 3: Object.prototype.toString.call()
console.log(Object.prototype.toString.call(arr) === '[object Array]'); // true
```

## 2. Замикання та область видимості

Питання: Що виведе наступний код?

```
for (var i = 0; i < 5; i++) {
  setTimeout(() => {
    console.log(i);
  }, 1000);
}
```

```

}

// Рішення з використанням let
for (let i = 0; i < 5; i++) {
  setTimeout(() => {
    console.log(i); // Виведе 0, 1, 2, 3, 4
  }, 1000);
}

// Рішення з використанням замикавання
for (var i = 0; i < 5; i++) {
  (function(j) {
    setTimeout(() => {
      console.log(j); // Виведе 0, 1, 2, 3, 4
    }, 1000);
  })(i);
}

```

#### Пояснення:

- При використанні **var** всі ітерації будуть посилатися на останнє значення **i**
- **let** створює блочну область видимості
- Негайно викликана функція створює замикавання для кожної ітерації

### 3. Прототипи та наслідування

Питання: Реалізуйте наслідування без використання **class**

```

// Функція-конструктор
function Animal(name) {
  this.name = name;
}

Animal.prototype.speak = function() {
  console.log(`${this.name} видає звук`);
};

// Наслідування
function Dog(name, breed) {
  Animal.call(this, name); // Виклик батьківського конструктора
  this.breed = breed;
}

// Встановлення прототипу
Dog.prototype = Object.create(Animal.prototype);
Dog.prototype.constructor = Dog;

// Перевизначення методу
Dog.prototype.speak = function() {
  console.log(`${this.name} гавкає`);
}

```

```
};

const dog = new Dog('Шарик', 'Вівчарка');
dog.speak(); // "Шарик гавкає"
```

## 4. Асинхронне програмування

Питання: Реалізуйте функцію послідовного виконання промісів

```
function sequentialPromises(promises) {
  return promises.reduce((chain, promise) => {
    return chain.then(result =>
      promise.then(value => [...result, value])
    );
  }, Promise.resolve([]));
}

// Приклад використання
const promise1 = () => new Promise(resolve => setTimeout(() => resolve(1), 1000));
const promise2 = () => new Promise(resolve => setTimeout(() => resolve(2), 500));
const promise3 = () => new Promise(resolve => setTimeout(() => resolve(3), 100));

sequentialPromises([promise1, promise2, promise3])
  .then(console.log); // [1, 2, 3]
```

Питання: Реалізуйте функцію `Promise.all` з нуля

```
function customPromiseAll(promises) {
  return new Promise((resolve, reject) => {
    if (!Array.isArray(promises)) {
      return reject(new TypeError('Аргумент має бути масивом'));
    }

    const results = new Array(promises.length);
    let completedPromises = 0;

    promises.forEach((promise, index) => {
      Promise.resolve(promise)
        .then(value => {
          results[index] = value;
          completedPromises++;

          if (completedPromises === promises.length) {
            resolve(results);
          }
        })
    });
  });
}
```

```

        })
        .catch(reject);
    });

    if (promises.length === 0) {
        resolve(results);
    }
});
}

// Приклад використання
const promises = [
    Promise.resolve(1),
    Promise.resolve(2),
    Promise.resolve(3)
];

customPromiseAll(promises)
    .then(console.log); // [1, 2, 3]

```

## 5. Робота з об'єктами

Питання: Глибоке клонування об'єкта

```

function deepClone(obj) {
    // Обробка примітивів та null
    if (obj === null || typeof obj !== 'object') {
        return obj;
    }

    // Обробка Date
    if (obj instanceof Date) {
        return new Date(obj.getTime());
    }

    // Обробка масивів
    if (Array.isArray(obj)) {
        return obj.map(deepClone);
    }

    // Обробка об'єктів
    const clone = {};
    for (let key in obj) {
        if (Object.prototype.hasOwnProperty.call(obj, key)) {
            clone[key] = deepClone(obj[key]);
        }
    }

    return clone;
}

```

```
// Приклад використання
const original = {
  a: 1,
  b: [1, 2, 3],
  c: { d: 4 },
  e: new Date()
};

const cloned = deepClone(original);
console.log(cloned !== original); // true
```

## 6. Функціональне програмування

Питання: Реалізуйте каррування функції

```
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn.apply(this, args);
    } else {
      return function(...moreArgs) {
        return curried.apply(this, args.concat(moreArgs));
      }
    }
  };
}

// Приклад використання
function sum(a, b, c) {
  return a + b + c;
}

const curriedSum = curry(sum);
console.log(curriedSum(1)(2)(3)); // 6
console.log(curriedSum(1, 2)(3)); // 6
console.log(curriedSum(1)(2, 3)); // 6
console.log(curriedSum(1, 2, 3)); // 6
```

## 7. Продуктивність та оптимізація

Питання: Напишіть функцію мемоізації

```
function memoize(fn) {
  const cache = new Map();

  return function(...args) {
    const key = JSON.stringify(args);
```

```

        if (cache.has(key)) {
            return cache.get(key);
        }

        const result = fn.apply(this, args);
        cache.set(key, result);
        return result;
    };
}

// Приклад використання
function fibonacci(n) {
    if (n <= 1) return n;
    return fibonacci(n - 1) + fibonacci(n - 2);
}

const memoizedFibonacci = memoize(fibonacci);
console.time('Без мемоізації');
fibonacci(35);
console.timeEnd('Без мемоізації');

console.time('З мемоізацією');
memoizedFibonacci(35);
console.timeEnd('З мемоізацією');

```

## Висновок

Поради для підготовки до співбесіди:

- Вивчайте основи мови глибоко
- Практикуйте розв'язання задач на платформах на кшталт LeetCode
- Розумійте, як працюють механізми мови "під капотом"
- Вмійте пояснити своє рішення
- Не бійтеся визнавати, що чогось не знаєте

---

Удачі на співбесіді!